

Software Engineering

Requirement Engineering

Okba Tibermachine

National School of Artificial Intelligence

October 22, 2025

Many slides and concepts in this presentation are taken from lecture materials prepared by Prof. Imed Boucherika (SE lectures, 2024, ENSIA)

Outline :

- Requirements Engineering
- Definitions
- Categorization of Requirements
- Requirements Engineering Process
- Requirements Gathering : Tools, Techniques..
- Requirements Analysis
- Requirements Documentation
- Validation and Verification
- Recommended Principles of RE
- SRS Assignment

Requirement Engineering

Requirements Engineering

What's a requirement ?

- Something required, wanted, needed. (stipulated as necessary to be done, made, or provided.)

IEEE Standard Glossary of SE Terminology

A software requirement is a condition or capacity needed by a user to solve a problem or achieve an objective.

- A condition or capability that must be met or possessed by a system or system component to satisfy a contract, standard, specification, or other formally imposed document.

Requirements Engineering

Requirements Engineering

Definition

It is a systematic process of developing the requirements through an iterative way of discovering, documenting and managing a set of requirements for a software system in addition to the constraints under which it operates.

*Requirements Engineering is complete sub-field in Software Engineering.
It is taught as a separate module at many universities*

Requirements Engineering

Objectives of Requirements Engineering

- To decide precisely what to build and document the results.
- To produce a set of system/software requirements that, as far as possible, is complete, consistent, relevant and reflects what the customer actually want

Requirements Engineering

Objectives of Requirements Engineering

- To decide precisely what to build and document the results.
- To produce a set of system/software requirements that, as far as possible, is complete, consistent, relevant and reflects what the customer actually want

Can we develop a Software Product without conducting the requirements engineering ?

Requirements Engineering

Why RE is extremely important :

- **Comprehension:** People do not know what they want
- **Communication:** Software requirements are difficult to communicate effectively in an informal way.
- **Protection for your company :** There are always changes, but there are riskier changes that can lead the project to go over budget.
- **Experience ?**

Requirements Engineering

Why RE is extremely important :

From Experience :

- Requirements are the biggest software engineering problem for software developers of large and complex systems.
- The failure to develop and document good requirements specifications is the major cause of software development failures. (40 - 60 % of all defects)

Requirements Engineering

Types of the Requirements:

Functional Requirements (FR)

- What the software product **shall do**
- "The system shall do"
- A system/software requirement that specifies a function that a system/software system or software components must be capable of performing.
- Defines the behavior of the system/software.

Requirements Engineering

Types of the Requirements:

Functional Requirements

Functionalities to be implemented

Case Study : "Quick Cup" Coffee Ordering App

- Browse the coffee menu and view drink prices
- Add, customize, and remove drinks from the cart
- Place an order and receive an order confirmation number
- Barista view to see and update incoming orders
- Update order status: "In Progress" → "Ready for Pickup"
- (Future feature) Apply time-based promotions like "Happy Hour"

Requirements Engineering

Types of the Requirements:

Non-Functional Requirements (NFR)

- Qualities the product needs to have.
- "The system needs to be"
- A software requirement that describes not what the software will do, but how the software will do it.
- Qualities and Constraints on how functional requirements are implemented

Requirements Engineering

Types of the Requirements:

Non-Functional requirements

- Conditions, constraints or quality requirements
- Constraints on how these functional requirements are implemented

Requirements Engineering

Types of the Requirements:

Non-Functional requirements

Case Study : "Quick Cup" Coffee Ordering App

- **Usability:** The app must be simple and intuitive for students to place an order within 3 clicks.
- **Performance:** Orders and menus should load in less than 2 seconds, even during rush hours.
- **Reliability:** The system must not lose or duplicate orders, even if the network connection is unstable.
- **Maintainability:** It should be easy for developers to add new drinks or seasonal offers later.

Requirements Engineering

Types of the Requirements:

Non-Functional requirements

Case Study : "Quick Cup" Coffee Ordering App

- ...
- **Security:** Sensitive data (e.g., user information, order IDs) must be protected from unauthorized access.
- **Availability:** The system should operate during shop hours with minimal downtime.
- **Scalability:** The app should handle increasing numbers of student users during peak times.

Requirements Engineering

Types of the Requirements:

Non-Functional requirements

Constraints :

Define the Non-Functional aspects of a system or component, such as restrictions on technology, resources or techniques to be used

Example :

- Software needs to run on Windows, Linux and iOS

Requirements Engineering

Types of the Requirements:

Non-Functional requirements

Non-functional requirements are very difficult :

- To Quantify and to state precisely
- → Imprecise requirements are difficult to verify.

Requirements Engineering

Types of the Requirements:

Non-Functional

Non-functional

Example Software must be easy to use : how to verify that this requirement is met ?

- To Quantify
- → Imprecise requirements are difficult to verify.

Requirements Engineering

Types of the Requirements:

Non-Functional Requirements

Instead of expressing a non-functional requirement as a **Goal**

Example, ease of use which is a general intention of the user

It is better to write them as a **Verifiable non-functional requirement**

A statement using some measure that can be objectively tested.

Requirements Engineering

Types of the Requirements:

Non-Functional requirements

Goal : The system should be easy to use by medical staff and should be organized in such a way that user errors are minimized.

Verifiable/Testable Statement : Medical staff shall be able to use all the system functions after four hours of training. After this training, the average number of errors made by experienced users shall not exceed two per hour of system use.

Requirements Engineering

Types of the Requirements: Non-Functional requirements

Speed

- Processed Transactions/Second
- Response Time
- Screen refresh time

Ease of use

- Training Time
- Duration to perform a task
- Steps/Clicks/Mouse Distance ..

Portability

- Number of target systems
- Percentage of target dependent statements

Reliability

- Mean time to failure
- Rate of failure occurrence
- Availability

Size

- Megabytes

Scaling

- Speed when increasing data
- Speed when 1000.. users access at the same time

Robustness

- Time to restart after failure
- Probability of data corruption

- Migrate system to another

Requirements Engineering

Types of the Requirements :

System requirements vs Software Requirements vs Hardware Requirements

- Software requirements = for the software components.
- System requirements = for the system as a whole. It includes
 - Software Requirements
 - Hardware Requirements

Requirements Engineering

Types of the Requirements :

System requirements vs Software Requirements vs Hardware Requirements

Information systems

- Requirements engineering is primarily software requirements engineering.

Embedded systems

- Requirements engineering involves both hardware and software requirements engineering.

Requirements Engineering

Types of the Requirements :

Abstract Requirements vs Detailed Requirements

If a company wants to hire another software development company to build their large software project, it must define its needs in a sufficiently abstract way that a solution is not predefined.

The requirements must be written so that several contractors can bid for the contract, offering, perhaps, different ways of meeting the client organization's needs.

Once a contract is awarded, the contractor must write a system definition for the client in more detail so that the client understands and can validate what the software will do.

RE process

Requirements Engineering Process

Requirements Engineering Process :

A set of activities and tasks that needs to be performed in a structured way.

When engineering the process, the following questions need to be addressed:

- The structuring or schedule of these activities.
- Who is responsible for each activity.
- The inputs and outputs to/from the activity.
- The tools used to support the process.
- The output of the process (RE)

Requirements Engineering Process

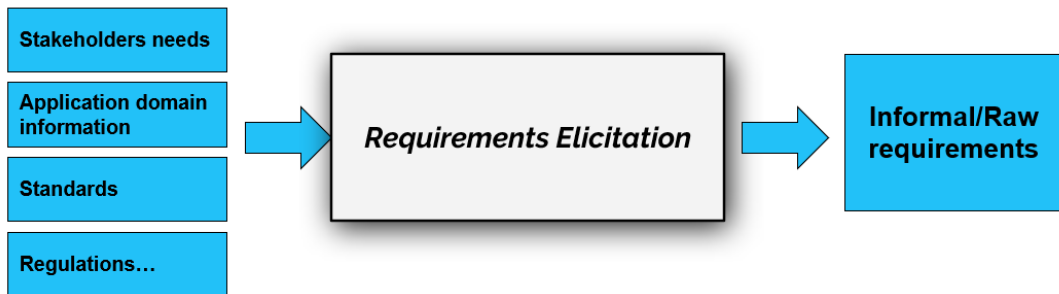
Requirements Engineering Process :

1. Requirements Elicitation
2. Requirements Analysis
3. Requirements Documentation
4. Validation

Requirements Engineering Process

Activity 1 : Requirements Elicitation

It is the process of discovering, gathering and understanding the requirements from the stakeholders and other relevant sources.



Requirements Engineering Process

Activity 1 : Requirements Elicitation

Defining who are the stakeholders ?

Stakeholder : is any entity that has a vested interest in the system to be built.

Type of Stakeholders :

- Internal : Project Manager, Team members (developers, testers....)
- External : Customer, end-user, supplier, government,...

Requirements Engineering Process

Activity 1 : Requirements Elicitation

Objective

Discover and understand the needs, expectations, and constraints of all stakeholders.

Requirements Engineering Process

Activity 1: Requirements Elicitation

Sources of Requirements:

- **Goals and Problems:** Identify why the system is needed and what problem it aims to solve.
- **Domain Knowledge:** Gather information about the business rules, concepts, and existing processes.
- **Stakeholders:** Consult clients, end-users, domain experts, and regulators.
- **Organizational Environment:** Understand workflows, policies, and validation or approval procedures.
- **Operational Environment:** Consider where and how the system will be used (e.g., devices, physical context).

Outcome: A clear, shared understanding of user needs that forms the basis for requirement specification.

Requirements Engineering Process

Activity 1 : Requirements Elicitation

How to elicit requirements

It is not :

- Going to the end-user/customer and just ask questions in an improvised way
- It would be a waste of time (Customer's time)
- The customer would feel that you are totally unprofessional and lack expertise.

How ?

Requirements Engineering Process

Activity 1 : Requirements Elicitation

How ?

A fully planned and organized process that requires :

- Have knowledge about:
 - Application domain
 - Organizational and Business process
 - Problem
- Choose an elicitation technique and prepare (Construct your questions...)
- Get as much information from the end-users as possible about their needs, constraints

Requirements Engineering Process

Activity 1: Requirements Elicitation

Techniques for Gathering Requirements:

- **Interviews:** Structured (closed) or unstructured (open) discussions to explore stakeholder needs.
- **Scenarios and Use Cases:** Describe typical user interactions to clarify goals and expectations.
- **Prototyping:** Build mock-ups or early versions of the system to refine and validate requirements.
- **Observation (Ethnography):** Observe users in their real work environment to uncover hidden needs.

Requirements Engineering Process

Activity 1: Requirements Elicitation

Techniques for Gathering Requirements:

-
- **Joint Application Development (JAD):** Collaborative workshops involving users and developers.
- **Questionnaires and Surveys:** Useful when dealing with large or distributed user groups.
- **Document Analysis:** Review existing systems, manuals, and business processes for background knowledge.

Outcome: A set of captured and organized stakeholder needs, observations, and preliminary requirements, ready for validation and formal specification in the next phase.

Requirements Engineering Process

Activity 1: Requirements Elicitation - Common Problems (Part 1)

Problem 1: Stakeholders don't know what they want

- *Example:* "We need a user-friendly interface" - but can't specify what makes it user-friendly
- Often only realize what they wanted after seeing what they don't want

Requirements Engineering Process

Activity 1: Requirements Elicitation - Common Problems (Part 1)

Problem 1: Stakeholders don't know what they want

- *Example:* "We need a user-friendly interface" - but can't specify what makes it user-friendly
- Often only realize what they wanted after seeing what they don't want

Problem 2: Stakeholders express requirements in their own terms

- *Example:* Doctor says "The system must be fast" (means < 2 sec response)
- Business says "Fast" (means processed by end of day)
- Technical debt: terminology mismatch leads to wrong implementation

Requirements Engineering Process

Activity 1: Requirements Elicitation - Common Problems (Part 1)

Problem 1: Stakeholders don't know what they want

- *Example:* "We need a user-friendly interface" - but can't specify what makes it user-friendly
- Often only realize what they wanted after seeing what they don't want

Problem 2: Stakeholders express requirements in their own terms

- *Example:* Doctor says "The system must be fast" (means <2 sec response)
- Business says "Fast" (means processed by end of day)
- Technical debt: terminology mismatch leads to wrong implementation

Problem 3: Different stakeholders have conflicting requirements

- *Example:* Marketing wants "easy signup, minimal fields"
- Security team wants "comprehensive verification, multiple authentication steps"
- Both are right from their perspective!

Requirements Engineering Process

Activity 1: Requirements Elicitation - Common Problems (Part 2)

Problem 4: Stakeholders unconvinced of need for new system

- *Example:* "Our Excel spreadsheets work fine, why do we need a database?"
- Resistance to change leads to incomplete requirements gathering

Requirements Engineering Process

Activity 1: Requirements Elicitation - Common Problems (Part 2)

Problem 4: Stakeholders unconvinced of need for new system

- *Example:* "Our Excel spreadsheets work fine, why do we need a database?"
- Resistance to change leads to incomplete requirements gathering

Problem 5: Stakeholders give unnecessary information

- *Example:* "The button should be blue because my daughter likes blue"
- Mixing personal preferences with actual business requirements

Requirements Engineering Process

Activity 1: Requirements Elicitation - Common Problems (Part 2)

Problem 4: Stakeholders unconvinced of need for new system

- *Example:* "Our Excel spreadsheets work fine, why do we need a database?"
- Resistance to change leads to incomplete requirements gathering

Problem 5: Stakeholders give unnecessary information

- *Example:* "The button should be blue because my daughter likes blue"
- Mixing personal preferences with actual business requirements

Problem 6: Organizational and political factors

- *Example:* Department head insists on features that protect their budget
- Hidden agenda: "This system must require 3 approval levels" (to maintain control)

Requirements Engineering Process

Activity 1: Requirements Elicitation - Common Problems (Part 2)

Problem 4: Stakeholders unconvinced of need for new system

- *Example:* "Our Excel spreadsheets work fine, why do we need a database?"
- Resistance to change leads to incomplete requirements gathering

Problem 5: Stakeholders give unnecessary information

- *Example:* "The button should be blue because my daughter likes blue"
- Mixing personal preferences with actual business requirements

Problem 6: Organizational and political factors

- *Example:* Department head insists on features that protect their budget
- Hidden agenda: "This system must require 3 approval levels" (to maintain control)

Problem 7: Requirements change during analysis

- *Example:* New competitor launches a feature mid-project
- Regulation changes (GDPR, accessibility laws) force requirement updates

Requirements Engineering Process

Activity 1: Requirements Elicitation - Common Problems (Part 2)

Problem 4: Stakeholders unconvinced of need for new system

- *Example:* "Our Excel spreadsheets work fine, why do we need a database?"
- Resistance to change leads to incomplete requirements gathering

Problem 5:

- *Example:*
- Mixing

Bigger Problem :
Implicitness and Assumptions

Problem 6: Organizational and political factors

- *Example:* Department head insists on features that protect their budget
- Hidden agenda: "This system must require 3 approval levels" (to maintain control)

Problem 7: Requirements change during analysis

- *Example:* New competitor launches a feature mid-project
- Regulation changes (GDPR, accessibility laws) force requirement updates

Requirements Engineering Process

Activity 1 : Requirements Elicitation

Problems with the Elicitation of Requirements :

Implicitness and Assumptions

- Stakeholders do not talk about certain requirements because they assume they are common sense to everyone (including the developers)
- Developers make assumptions about how end-users think and about their needs.

Requirements Engineering Process

Activity 1: Requirements Elicitation

Problem: Implicitness and Assumptions (cont.)

- **Stakeholder Assumption Example:** The client says, “Of course the system will save data automatically,” assuming this is obvious, but never explicitly states it as a requirement. → The developers implement manual saving only.
- **Developer Assumption Example:** Developers assume all users have constant internet access, but in reality, many use the app in areas with poor connectivity. → The system fails when offline.
- **Lesson:** Never assume “common sense.” All expectations must be made explicit, verified, and documented.

Requirements Engineering Process

Activity 1 : Requirements Elicitation: Questions to Ask

- Why do you need to build a new software ?
- Can you just use an Excel file ?
- Can you just use that existing software ?
- What problem are you expecting the software to solve ?
- Who will be using the software ? who are the actual users of the software ?
- Where the software will be used ?
- Given that the user X want to perform a task Y. Can we describe the scenario for him in doing that task ?
- How would you assess that the software product is a success ? What would you like to see at the end ?
- ... (including more why, when, where, what and how)

Requirements Engineering Process

Activity 2 : Requirements Analysis

The process of reviewing and analyzing the raw requirements in order to discover:

- Missing Requirements
- Overlapping Requirements
- Ambiguous Requirements
- Unrealistic Requirements
- Conflictual Requirements
- Inconsistent Requirements

And negotiating with the stakeholders to resolve any conflict, inconsistency and improving the interpretation of raw requirements

Requirements Engineering Process

Activity 2: Requirements Analysis

Goal: Examine the elicited requirements to detect issues, gaps, and inconsistencies before specification.

Typical Problems Found During Analysis:

- **Missing Requirements:** Some functionalities or constraints are not mentioned at all. *Example:* “The system prints reports” — but the format, data, or timing are unspecified.
- **Overlapping Requirements:** Different stakeholders express the same feature in multiple ways. *Example:* “Daily alerts” and “automatic reminders” refer to the same notification function.
- **Ambiguous Requirements:** Statements are vague or open to multiple interpretations. *Example:* “The system should be fast and user-friendly.” (What does “fast” mean?)

Requirements Engineering Process

Activity 2: Requirements Analysis (continued)

Further Issues Discovered During Analysis:

- **Unrealistic Requirements:** Requested features exceed technical, time, or budget constraints. *Example:* “Handle 1,000 transactions per second” on low-cost hardware.
- **Conflicting Requirements:** Different stakeholders want contradictory behaviors. *Example:* Managers need detailed logs; users demand minimal data for privacy.
- **Inconsistent Requirements:** Requirements contradict each other or use inconsistent terminology. *Example:* One states “Pay at pickup,” another says “Online payment only.”

Negotiate with stakeholders to resolve conflicts, clarify ambiguities, and consolidate consistent, feasible requirements.

Requirements Engineering Process

Activity 3 : Requirements Documentation

The process of developing a document that, clearly, precisely, formally and officially records each of the requirements of the software system.

This document is used to communicate the requirements to:

- Customers
- End-users
- Software developers.
- Project Managers
- Quality Assurance Engineers/Testers

The resulting artifact from this process is mostly a DRAFT version of the Software Requirements Specification (SRS) document

Requirements Engineering Process

Activity 3 : Requirements Documentation

Software Requirements Specification (SRS):

- Identifies the customer requirements by detailing the results of the elicitation , problem analysis and even modelling.
- Serves as foundation for the design, implementation and testing of the software.
- Provides guidance for future system maintenance
- It must be : Correct, Complete, Unambiguous, Consistent, Verifiable, Understandable, Modifiable, Traceable...

Requirements Engineering Process

Activity 3 : Requirements Documentation

Software Requirements Specification (SRS):

There is no best way to write requirements but there are recommendations to follow :

- IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications
- IEEE Std 1362-1998, Guide for Information Technology -System Design -Concept of Operations Document
- IEEE Std 1233-1998, Guide for Developing System Requirements Specification

Most requirements are written as natural language sentences supplemented with diagrams, tables of detailed information, formal or semi-formal descriptions.

Requirements Engineering Process

Activity 3 : Requirements Documentation

Structure of the Software Requirements Specification (SRS) document:

IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications

See provided examples

Table of Contents of an SRS

1. Introduction
 - 1.1 Purpose
 - 1.2 Scope
 - 1.3 Definitions, acronyms, and abbreviations
 - 1.4 References
 - 1.5 Overview
2. Overall description
 - 2.1 Product perspective
 - 2.2 Product functions
 - 2.3 User characteristics
 - 2.4 Constraints
 - 2.5 Assumptions and dependencies
3. Specific requirements (*the largest & most important part of an SRS, organized in different manner*)
- Supporting informations
 - Table of contents and Index
 - Appendixes (e.g. Samples I/O formats, background, description of problem to be solved by the software)

Volere Requirements Template

Volere (by Suzanne and James Robertson) is a requirements specification template and methodology that evolved later.

It is still document-based like IEEE 830, but more human-centered and flexible.

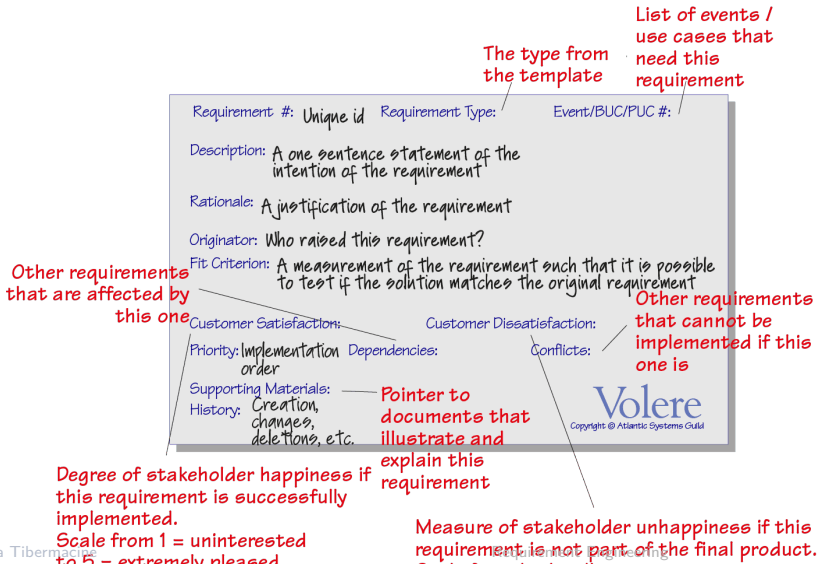
Key Ideas:

- Organizes requirements using a consistent structure (*fit criteria*, rationale, source, etc.).
- Encourages both business and technical perspectives.
- Supports iterative development — usable in both Waterfall and Agile contexts.

Relation to IEEE 830:

- Volere is a refined and modernized version of the IEEE 830 SRS concept.
- Provides a template that maps easily to IEEE 830 sections, adding guidance for quality, rationale, and stakeholder communication.

Requirements Engineering Process



Requirements Engineering Process

Activity 4 : Requirements Validation

Certifies that the requirements document is an acceptable description of the system to be implemented.

We finalize the SRS document at this stage with the stakeholders checking for :

- Completeness and consistency
- Requirements conflicts
- Ambiguous requirements
- Conformance to standards
- Technical errors

Requirements Engineering Process

Activity 4 : Requirements Validation

Certifies that the requirements document is an acceptable description of the system to be implemented.

We finalize the SRS document at this stage with the stakeholders checking for :

- Completeness and consistency
- Requirements conflicts
- Ambiguous requirements
- Conformance to standards
- Technical errors

Iteratively, send your PV/Document containing the analysed requirements during that meeting to the involved stakeholders after each meeting/interview/session

Recommended Principles of RE

01

Value-orientation : Ask the right questions

Focus on the real problem or need not the solution to the need

Requirements are a means to an end, not an end in itself!

Requirements engineering places a strong focus on the value creation of a development.

Example: If the requirement is “record patient information”, this is not the end — it is the means.

The real value is to ensure accurate and accessible medical records for better patient care.

Recommended Principles of RE

02

Stakeholder : Involve all & Engage Early and Often

RE is about satisfying the stakeholders' desires and needs!

In order to understand the wishes and needs of stakeholders, stakeholder management is a core task in requirements engineering.

Different roles, different age, different education levels....your system must satisfy their needs

Recommended Principles of RE

03

WRITE : clear and concise requirements

Don't pretend you have a solid memory

Ensure PEN and Notebook to write the requirements using **SMART** method :

Specific, **M**easurable, **A**chievable, **R**elevant and **T**ime-bound.

04

Context is Important

Systems cannot be understood in isolation!

Systems are embedded in a context. The system context describes the interaction of a system with its environment.

When needed for a requirement, if there is a context, make sure to write it down.

Recommended Principles of RE

05

Ensure better operational management process

Traceability links : Ensure there is a link between the requirement to the design, development, testing and integration. If there is a change, it can be easily tracked through the full lifecycle.

Versions & Changes : Track all changes to ensure they justified, approved, communicated and implemented.

Recommended Principles of RE

06

ReAdapt to your Process Model/Methodologies

If plan driven, make sure you complete all the requirements and validate them

For Agile models, embrace always evolving and potentially new requirements into the product backlog.

Recommended Principles of RE

07

Non-Functional Requirements can be critical

Don't Ignore them, they can be vital to the success of the product..

08

Clearly set the "Done Definition" or Acceptance Criteria

Describe clearly the acceptance criteria when a requirement is considered completed.

This ensure better and common understanding between developers, testers and stakeholders.

Recommended Principles of RE

09

Cyclic : Problem - Requirement - Solution

Having understood the problem and the requirement.

Can be good to speak out and brainstorm solutions with the stakeholders in order to bring new requirements, problems and constraints (in their presence)

10

Use Visual/Graphical Diagrams to Communicate

Use use case diagram, process flow and even UI mockups to simplify and provide visual representation of the requirements.

Recommended Principles of RE

Prioritize the Requirements

Prioritize the requirements based on :

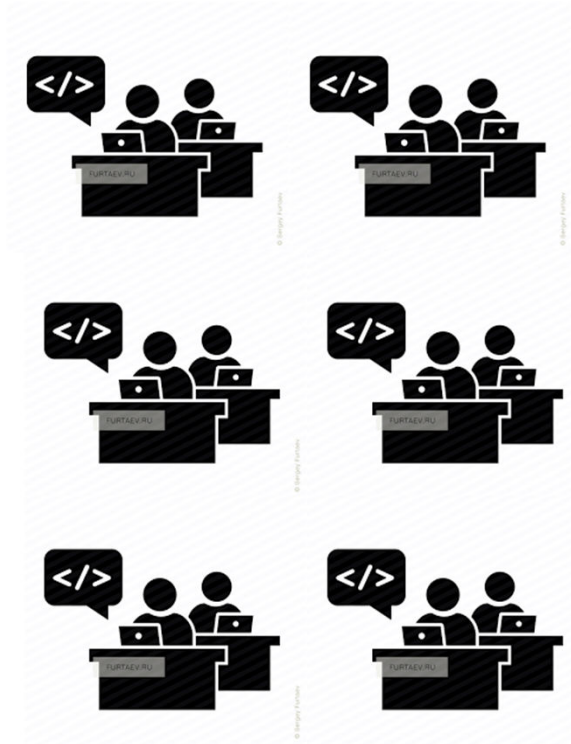
- Business Value
- Cost
- Effort
- Technical Complexity
- Risk

You may use existing notation/methods for the prioritization including MoSCoW (M : Must have, S: Should, C: Could, W : Won't have..

All stakeholders can contribute to prioritize the requirements.

User Stories

Before ...

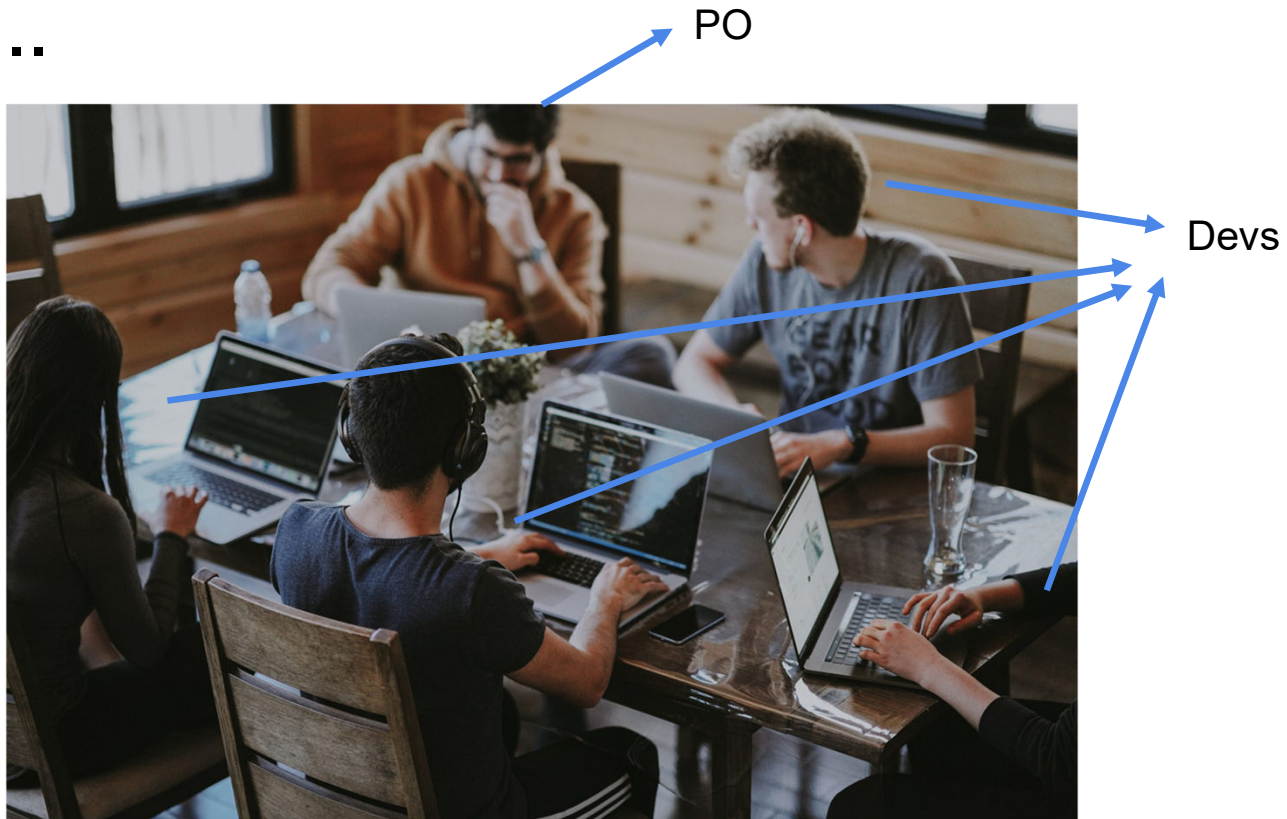


Analysts

(PRD: Product Requirements Document)

Programmers
(Software Agency)

Today ...



The Product Owner works directly with developers to explain requirements

User Stories = 3C's

- **C**ard + **C**onversations + **C**onfirmation
- Card: reminder for conversations during sprint
- Conversations: as in previous slide
- Confirmation: tests the PO will perform to verify the story's implementation; also called acceptance tests (or acceptance criteria)

Example: Online Bookstore

- Card: As a user, I want to buy a book
- Conversations: PO explains payment and delivery methods, discount options, etc
- Confirmation:
 - Test purchase with a discount code
 - Test payments with Visa and Mastercard
 - Test deliveries with both standard and express options

Writing Workshop (or Inception)

- When: project beginning
- Participants: key users
- Goal:
 - Define what the software will do $\Rightarrow$ initial list of stories
 - Define what the software will not do

Setup Sprint

- Before starting, some teams also conduct a "setup sprint"
- Goal: set up the development environment and tools

Format of User Stories

- As a [type of user],
I want to [perform an action with the system]
So That [Benefit/Value]

Example: Library Management System

Types of Users

- Students
- Instructors
- Staff members

Students' Stories

- As a student, I want to borrow books so that I can access the materials I need for my studies.
- As a student, I want to return borrowed books so that other students can use them and I can avoid late penalties.
- As a student, I want to renew my book loans so that I can keep the books longer when I still need them.
- As a student, I want to search for books so that I can quickly find the resources relevant to my courses or research.
- As a student, I want to reserve books that are currently on loan so that I can borrow them as soon as they become available.
- As a student, I want to receive notifications about new acquisitions so that I can stay informed about the latest materials in my field of interest.

Instructors' Stories

- As an instructor, I want to borrow books for an extended period so that I can use them throughout the semester for teaching and research purposes.
- As an instructor, I want to recommend books for acquisition so that the library can expand its collection with resources that benefit students and academic programs.
- As an instructor, I want to donate books to the library so that other readers can access valuable materials I no longer need.
- As an instructor, I want to return books to other campus libraries so that inter-campus sharing and resource circulation remain efficient and up to date.

Staff Stories

- As a staff member, I want to register new users so that they can access library services and borrow books.
- As a staff member, I want to add new books to the system so that the library catalog remains complete and up to date.
- As a staff member, I want to remove damaged books from the system so that users only see and borrow books in good condition.
- As a staff member, I want to access statistics about the collection so that I can monitor usage, plan acquisitions, and improve resource management.
- As a staff member, I want the system to send reminders to users with overdue books so that borrowed materials are returned on time and availability is maintained.
- As a staff member, I want the system to charge fines for late book returns so that users are encouraged to respect borrowing deadlines and ensure fair access for everyone.

INVEST

- **Independent:** The user story should stand alone, meaning it can be developed and delivered without relying on other stories.
- **Negotiable:** The user story isn't a rigid contract; it's open to discussion and refinement based on feedback and collaboration.
- **Valuable:** The user story must provide clear value to the user or business by addressing a real need or delivering an improvement.
- **Estimable:** The scope and content should be clear enough for the team to estimate the effort required to complete the user story.
- **Small:** The user story should be sized appropriately — small enough to be completed within a single sprint or workflow iteration.
- **Testable:** There must be clear acceptance criteria so that the user story can be tested and validated upon completion.

User Stories $\Leftrightarrow$ Functional Requirements

But how can we define Non-Functional Requirements?

Ways to Represent NFRs in Scrum

Option 1: As Acceptance Criteria within User Stories

- Attach NFRs directly to user stories that they affect.

Example:

User Story: As a customer, I want to log in so I can access my dashboard.

Acceptance Criteria:

- Login must complete within 2 seconds. (*Performance*)
- Passwords must be encrypted using bcrypt. (*Security*)

Best for story-specific NFRs.

Ways to Represent NFRs in Scrum

Option 2: As Separate Technical Stories

- If the NFR requires dedicated effort or infrastructure.

Example:

- *As a DevOps engineer, I want to set up an automated load testing pipeline so that we can ensure performance compliance.*

Best for NFRs that require engineering work.

Ways to Represent NFRs in Scrum

Option 3: As Definition of Done (DoD) Items

- Incorporate common NFRs into the team's Definition of Done.

Example DoD items:

- Code reviewed and passes security scan
- API latency under 500 ms
- Unit test coverage $\geq 80\%$

Best for recurring, system-wide NFRs.

Acceptance Criteria (AC)

Purpose: To define **when a specific user story** is complete and acceptable to the **Product Owner**.

Scope:

- Applies to a **single user story**.
- Written **before or during refinement** of that story.
- Ensures the story meets **functional and relevant non-functional expectations**.

Example:

User Story:

As a customer, I want to reset my password so that I can regain access if I forget it.

Acceptance Criteria:

- User receives an email with a reset link.
- Link expires after 24 hours.
- Password must contain at least 8 characters and one symbol.
- Reset process completes in under 3 seconds.

If all AC are met → the story is accepted.

Definition of Done (DoD)

- **Purpose:** To define **what “done” means for *any* backlog item** (user story, bug, task, etc.) within the team.
- **Scope:**
 - Applies **globally to all items** the Scrum Team delivers.
 - Ensures **quality, consistency, and completeness** of every increment.
 - Typically defined **by the whole Scrum Team**, and it evolves over time.

Example: Definition of Done (DoD)

- Code reviewed and merged into main branch.
- Unit tests written and passed (coverage $\geq 80\%$).
- Deployed to staging and passes regression tests.
- Documentation updated.
- Meets security and performance standards.

If all DoD items are met → the increment is potentially shippable.

Think of it like this:

- **Acceptance Criteria** = “Have we built the *right* thing for this story?”
- **Definition of Done** = “Have we built it *well enough* to deliver?”

Acceptance Tests (AT)

- **Purpose:**
- To **verify** (by execution) that the Acceptance Criteria are actually met.
- **Nature: Executable** → often automated or written in test scripts (BDD style, e.g., *Given / When / Then*).

Written by **QA/testers** or collaboratively by the team.

Serve as the **proof** that the AC are satisfied.

Example (for the same story):

- **Acceptance Test 1:**
Given a registered user,
When they request a password reset,
Then they receive an email with a reset link.
- **Acceptance Test 2:**
Given a user with an expired link,
When they click it,
Then the system shows an error message “This link has expired.”

Exercises about User Stories

Exercise

3. Consider an online forms application, similar to Google Forms. Suppose the following user story:

“As a form creator, I want to close my forms to stop receiving new responses”

Write an **acceptance test** for this story.

Possible answer:

- Story: As a form creator, I want to close my forms to stop receiving new responses
- Acceptance test:
 - Create a form (or select an existing form)
 - Submit two responses (as a regular user)
 - Close the form to receive responses
 - Try to submit a new response and check that it will not be possible

4. Consider an e-commerce system with the following user story:

“As a customer, I want to view the status of my purchases”

Write an **acceptance test** for this story.

Possible answer:

- Story: As a customer, I would like to list the status of my purchases
- Acceptance test:
 - List purchases as in “track_status.xls” (this is an example of user's purchases, with all the possible data and statuses)
 - When clicking a purchase, its details should be shown as in the “purchase data” tab of the spreadsheet
 - It should be possible to sort the list by date, value, and status
 - We do not need support for searching and filtering purchases

5. Suppose you are the technical leader of a team and are responsible for defining its **done criteria**. List at least three checks you would recommend including.

Possible answer:

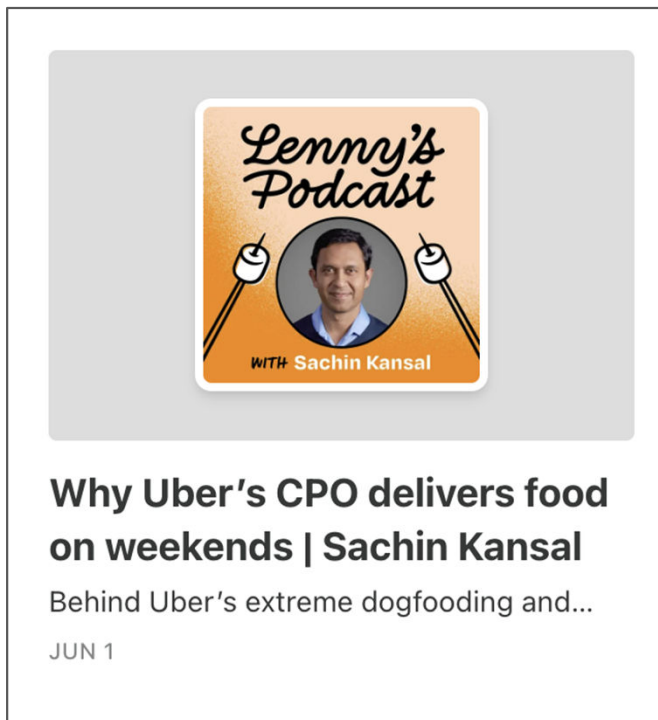
- Done criteria:
 - a. Test coverage $\geq 80\%$
 - b. Linter tool executed with no warnings
 - c. UX designer has approved any changes in the frontend
 - d. Code has been reviewed by at least one other team member
 - e. Code has been merged into the development branch

6. In Software Engineering, anti-patterns are ineffective solutions to recurring problems. Describe five anti-patterns for user stories—i.e., stories that are ineffective or violate good practices.

Dogfooding ("eat your own dog food")



Another Example: Uber



[link](#)

- 1–2 times a month works as a driver
- Goal: using the app in a car is different from using it in the office due to car speed, distance between the phone and the driver, etc.
- "This thing that was looking so great in an office setting, now maybe makes no sense."

Dogfooding Uber Eats

- The Uber office also has a cafeteria
- Therefore, developers can:
 - Place orders (customer view)
 - Receive orders (restaurant view)
 - Make deliveries (courier view)

Use Cases

Use Cases

- Detailed requirement specifications
- Rarely used in agile methods
- Describe an actor performing an operation with the system
- Two flows: normal flow and extensions
- Extensions:
 - Exceptions (or errors)
 - Additional details for specific steps

Transfer Values between Accounts

Actor: Bank Customer

Main Flow:

- 1 - Authenticate Customer
- 2 - Customer enters destination account and branch
- 3 - Customer enters the amount for transfer
- 4 - Customer specifies the transfer date
- 5 - System executes the transfer
- 6 - System asks if the customer wants to make another transfer

Extensions:

- 2a - If the account or branch is incorrect, system requests new account and branch
- 3a - If transfer amount exceeds current balance, system requests new amount
- 4a - Transfer date must be the current date or within one year in the future
- 5a - If the transfer date is the current date, system processes transfer immediately
- 5b - If the transfer date is in the future, system schedules the transfer

Important

- Use cases are not algorithms
- We are still defining the requirements:
 - Understanding and defining the system boundaries
 - Instead of proposing solutions (algorithms)

Example: Processing a Sale at the POS (Point-of-Sale)

Source: Craig Larman. Applying UML and Patterns. Pearson, 2004

Main Flow

1. Customer arrives at POS checkout with goods and/or services to purchase.
2. Cashier starts a new sale.
3. Cashier enters item identifier.
4. System records sale line item and presents item description, price, and running total. Price calculated from a set of price rules.
Cashier repeats steps 3-4 until indicates done.
1. System presents total with taxes calculated.
2. Cashier tells Customer the total, and asks for payment.
3. Customer pays and System handles payment.
4. System logs completed sale and sends sale and payment information to the external Accounting system (for accounting and commissions) and Inventory system (to update inventory).
5. System presents receipt.
6. Customer leaves with receipt and goods (if any).

Extensions: [we will show only two steps]

7a. Paying by cash:

1. Cashier enters the cash amount tendered.
2. System presents the balance due, and releases the cash drawer.
3. Cashier deposits cash tendered and returns balance in cash to Customer.
4. System records the cash payment.

7b. Paying by credit:

1. Customer enters their credit account information.
2. System sends payment authorization request to an external Payment Authorization Service System, and requests payment approval.
 - 2a. System detects failure to collaborate with external system:
 1. System signals error to Cashier.
 2. Cashier asks Customer for alternate payment.
3. System receives payment approval and signals approval to Cashier.
4. System records the credit payment, which includes the payment approval.
5. System presents credit payment signature input mechanism.
6. Cashier asks Customer for a credit payment signature. Customer enters signature.